



Republic of the Philippines
Tarlac State University
COLLEGE OF COMPUTER STUDIES
Tarlac City, Tarlac
Tel. No. (045) 6068173



Case Study: Simulation of CPU Scheduling Algorithm

**A Case Study Presented to
The Faculty of the Bachelor of Science in Computer Science
College of Computer Studies**

**In Partial Fulfillment
of the Requirements for the Course
Operating System**

Prepared by:

Pineda, Mary Jazlyn S.

Submitted to:

Mrs. Jo Anne G. Cura

Due Date: May 15, 2026



I. Introduction

CPU Scheduling in Operating Systems

Modern computing systems are built on the principle of multiprogramming — the ability to keep multiple processes in memory simultaneously and switch CPU execution among them to maximize utilization. At the core of this mechanism lies CPU scheduling, the process by which the operating system determines which process in the ready queue is granted access to the processor at any given time. Because the CPU is the most valuable and finite resource in a computer system, the manner in which it is allocated has a direct and measurable impact on system responsiveness, throughput, and fairness (Silberschatz et al., 2018).

Scheduling decisions are guided by algorithms that differ in their criteria, complexity, and behavioral trade-offs. These algorithms fall into two broad categories: non-preemptive, in which a process holds the CPU until it completes or blocks voluntarily, and preemptive, in which the operating system may forcibly reclaim the CPU mid-execution to serve a more urgent process (Stallings, 2018). Each approach carries distinct implications for performance metrics such as Waiting Time (WT), Turnaround Time (TAT), response time, and CPU utilization.

Understanding CPU scheduling is not merely an academic exercise. As Tanenbaum and Bos (2015) observed, the scheduler is one of the most consequential components of any operating system, as its decisions affect every process running on the machine. A poorly chosen or improperly implemented scheduling policy can lead to degraded performance, unresponsive systems, or the starvation of low-priority processes — all of which have real consequences in production environments.

Purpose of This Case Study

This case study was undertaken as a practical complement to the theoretical study of CPU scheduling in the Operating Systems course. The act of building a working simulation — one that accepts real process parameters, executes scheduling logic, and produces verifiable output — transforms abstract algorithmic concepts into tangible, observable behavior. As Arpaci-Dusseau and Arpaci-Dusseau (2018) argued, policy understanding is only complete when one can reason not just about what an algorithm does, but about how it behaves across varied and realistic workloads.

The simulation developed for this case study implements six scheduling algorithms in a fully interactive, web-based program built using HTML, CSS, and JavaScript. Users may define their own processes by specifying arrival times, burst times, and priority values, then observe the resulting Gantt chart and performance statistics in real time. This design makes the program both a learning tool and a demonstration of the principles studied throughout the course.

Objectives of the Program

The program developed for this case study is designed to fulfill the following specific objectives:

**Algorithm Simulation:**

Implement and correctly execute six CPU scheduling algorithms — FCFS, SJF, SRT, Round Robin, Priority Scheduling (Non-Preemptive and Preemptive), and Priority Scheduling with Round Robin — covering both preemptive and non-preemptive behaviors as defined in operating systems literature (Silberschatz et al., 2018).

User-Defined Input:

Accept a minimum of three user-specified processes, each with configurable Arrival Time, Burst Time, Priority value, and Time Quantum where applicable, allowing the simulation to reflect realistic and varied workloads.

Performance Metric Computation:

Calculate and display per-process Waiting Time (WT) and Turnaround Time (TAT), along with system-wide Average Waiting Time (AWT) and Average Turnaround Time (ATAT), providing a quantitative basis for evaluating and comparing algorithm behavior.

Visual Output:

Render a color-coded Gantt Chart for each simulation run, giving a clear visual representation of CPU allocation, process execution order, and idle periods across the timeline (Stallings, 2018).

Comparative Analysis:

Enable side-by-side comparison of algorithm results to draw informed conclusions about the relative efficiency, fairness, and suitability of each scheduling approach under different process configurations.



II. Body

A. First-Come, First-Served (FCFS)

How the Algorithm Works

First-Come, First-Served is the most straightforward CPU scheduling algorithm. It operates on a simple First-In, First-Out (FIFO) principle: processes are executed in the exact order in which they arrive in the ready queue. The first process to arrive is the first to receive CPU time, and it runs to full completion before the next process begins. No process is ever interrupted once it starts (Silberschatz et al., 2018).

In the simulation program, FCFS sorts all input processes by their arrival time and executes them sequentially. If a gap exists between the completion of one process and the arrival of the next, the Gantt chart renders an idle block to accurately reflect CPU downtime.

Type: Non-Preemptive

Advantages:

- Extremely simple to implement and understand.
- Guarantees that every process will eventually execute; no starvation occurs.
- Predictable execution order, which can simplify debugging and tracing.

Limitations:

- Highly susceptible to the convoy effect, where a single long process forces all subsequent shorter processes to wait, artificially inflating average waiting time.
- Does not account for urgency or process length, making it unsuitable for interactive or real-time systems (Tanenbaum & Bos, 2015).
- Average Waiting Time can be disproportionately high when burst times vary significantly across processes.



Figure 1. First-Come First-Served Scheduling Algorithm



Sample Test Case 1 — First-Come First-Served (FCFS)

✿ Process Input

NUMBER OF PROCESSES

4

✦ Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	4	3
P2	1	6	1
P3	2	3	4
P4	4	2	2

✦ Run Simulation

Figure 2. Process Input of FCFS Case 1



Figure 3. Gantt Chart of FCFS Case 1

✿ Process Statistics

PROCESS	ARRIVAL TIME	BURST TIME	FINISH TIME	TURNAROUND TIME	WAITING TIME
P1	0	4	4	4	0
P2	1	6	10	9	3
P3	2	3	13	11	8
P4	4	2	15	11	9

✦ AVERAGE WAITING TIME

5.00

✦ AVERAGE TURNAROUND TIME

8.75

Figure 4. Process Statistics of FCFS Case 1



In this test case, all four processes arrive very close together. P1 arrives first at $t=0$ and immediately occupies the CPU, running from $t=0$ to $t=4$. P2 arrives at $t=1$ but must wait until P1 finishes, then runs from $t=4$ to $t=10$. P3 and P4 similarly queue behind the processes already running. The convoy effect is observable here: P3, despite having a short burst time of only 3 units, must wait 8 units because it arrived behind the longer P2. P4 accumulates the worst waiting time of 9 units. This test case illustrates that FCFS does not consider burst time at all — only arrival order — which leads to disproportionately high waiting times for shorter processes that happen to arrive behind longer ones (Tanenbaum & Bos, 2015).

Sample Test Case 2 — First-Come First-Served (FCFS)



Figure 5. *Process Input of FCFS Case 2*



Figure 6. *Gantt Chart of FCFS Case 2*

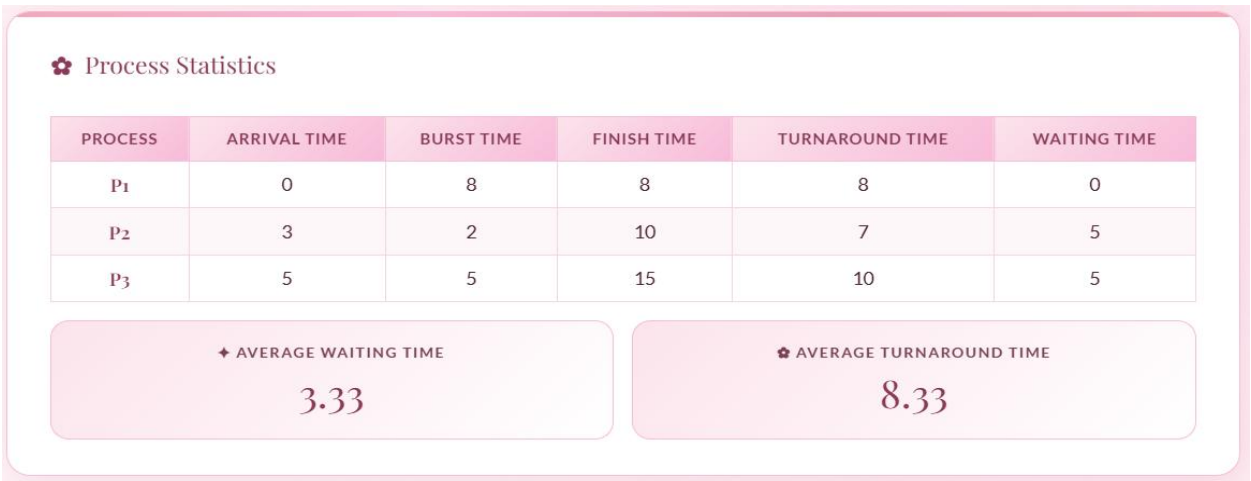


Figure 7. *Process Statistics of FCFS Case 2*



This test case with only three processes produces a lower average waiting time than Test Case 1 but still demonstrates the core weakness of FCFS clearly. P1, with a burst time of 8 units, monopolizes the CPU from $t=0$ to $t=8$. P2, which has the shortest burst time of only 2 units and arrived at $t=3$, is forced to wait 5 units before it can execute — entirely because it arrived after P1. This is a textbook example of the convoy effect: P2's short execution requirement is completely irrelevant to FCFS, which only considers arrival order. With fewer processes, the average metrics appear more favorable, but the structural unfairness of the algorithm toward short-burst processes that arrive late is just as present (Silberschatz et al., 2018).

B. Shortest Job First (SJF) — Non-Preemptive

How the Algorithm Works

Shortest Job First selects the process with the smallest burst time from among all processes that have arrived and are waiting in the ready queue at the moment the CPU becomes free. The selected process then executes to completion without interruption. When two or more processes share the same burst time, arrival time is used as a tiebreaker (Silberschatz et al., 2018).

In the simulation, SJF evaluates all available processes each time the CPU becomes idle and picks the one with the minimum burst time. Processes that have not yet arrived are not considered, which means the algorithm may still idle if the next available process has not arrived when the CPU becomes free.

Type: Non-Preemptive

Advantages:

- Mathematically proven to yield the minimum average waiting time among all non-preemptive algorithms for a given set of processes.
- Effectively eliminates the convoy effect by consistently prioritizing shorter jobs.
- Simple scheduling decision logic once burst times are known.

Limitations:

- Requires advance knowledge of each process's burst time, which is generally unavailable in real systems and must be estimated (Hansen, 2001).
- Longer processes risk starvation if a continuous stream of shorter processes keeps arriving.
- Not suitable for interactive environments where response time matters more than throughput.



Choose Scheduling Algorithm

FCFS
First-Come First-Served

SJF
Shortest Job First

SRT
Shortest Remaining Time

Round Robin
RR

Priority NP
Non-Preemptive

Priority P
Preemptive

Priority + RR
With Quantum

Figure 8. Shortest Job First Scheduling Algorithm

Sample Test Case 1 — Shortest Job First (SJF)

Process Input

NUMBER OF PROCESSES

4

Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	7	2
P2	2	3	1
P3	3	5	3
P4	5	1	4

Run Simulation

Figure 9. Process Input of SJF Case 1

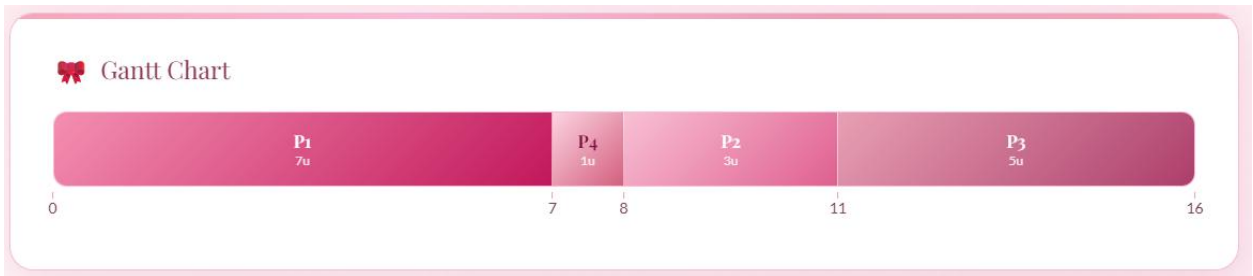


Figure 10. Gantt Chart of SJF Case 1

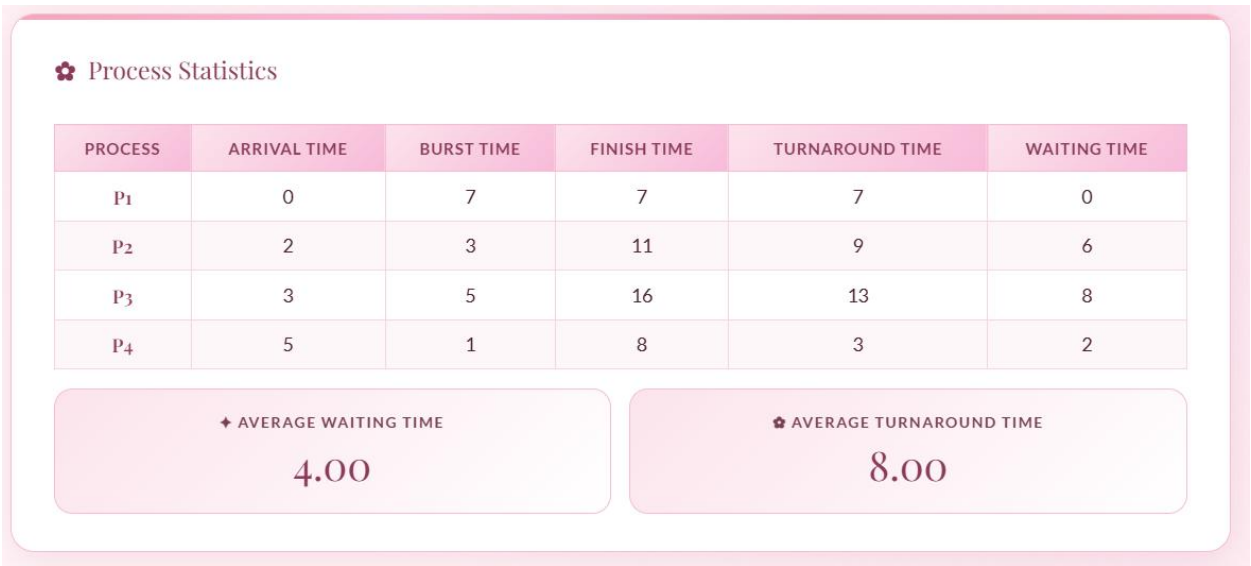


Figure 11. Process Statistics of SJF Case 1

At $t=0$, only P1 is available, so it runs first despite its relatively long burst time of 7 units. Once P1 finishes at $t=7$, three processes have already arrived: P2 (BT=3), P3 (BT=5), and P4 (BT=1). SJF selects P4 first because it has the shortest burst time of 1 unit, followed by P2 (BT=3), and finally P3 (BT=5). The ordering $P_4 \rightarrow P_2 \rightarrow P_3$ is exactly what SJF prescribes and results in noticeably lower average waiting and turnaround times compared to what FCFS would have produced with the same input. P4 particularly benefits, finishing in only 3 units of turnaround time despite arriving last among the three late processes (Silberschatz et al., 2018).

Sample Test Case 2 — Shortest Job First (SJF)

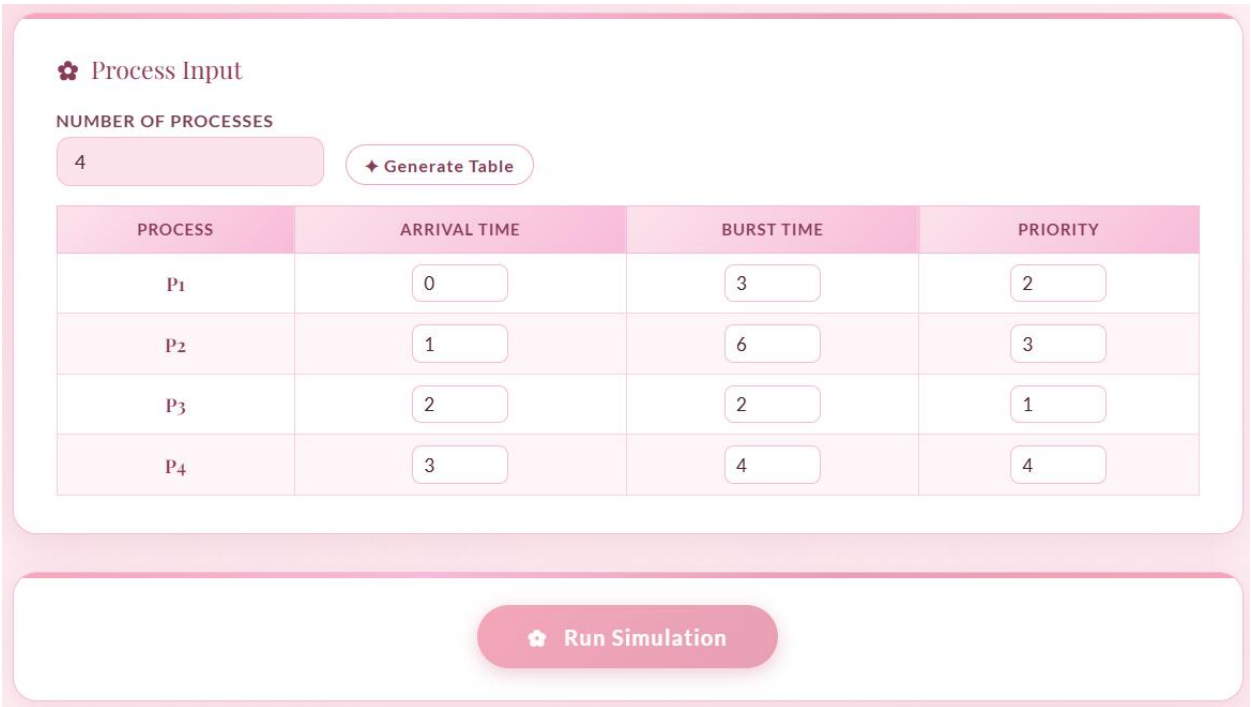


Figure 12. Process Input of SJF Case 2



Figure 13. *Gantt Chart of SJF Case 2*

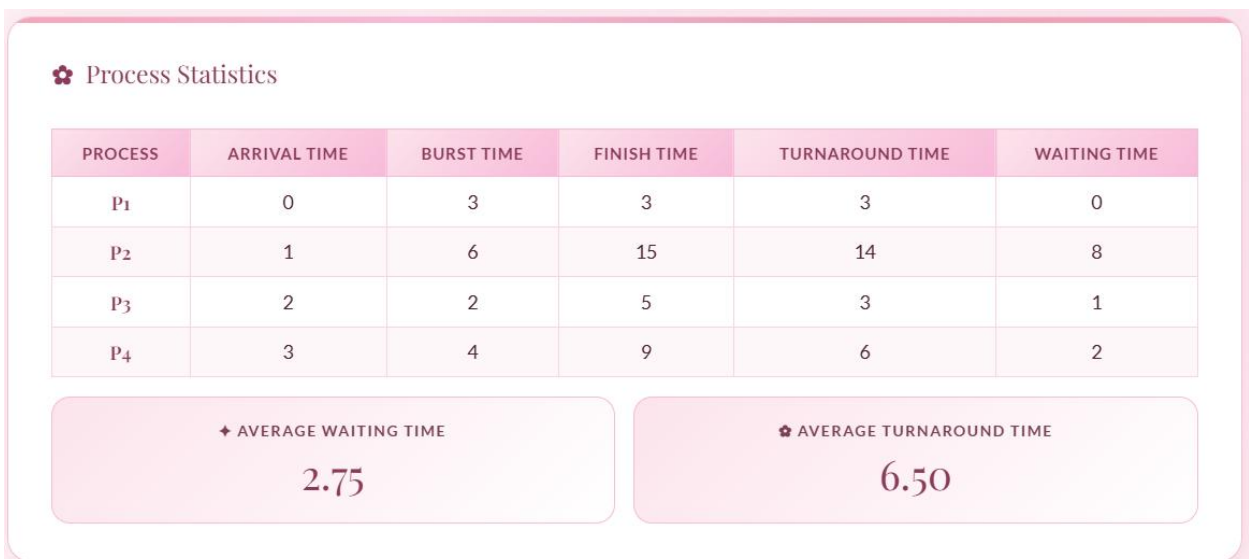


Figure 14. *Process Statistics of SJF Case 2*

Test Case 2 achieves the lowest average waiting time of all SJF tests at 2.75, demonstrating the algorithm's efficiency when processes arrive in a favorable order. P1 executes first at $t=0$ and finishes at $t=3$. At that point, P2 (BT=6), P3 (BT=2), and P4 (BT=4) are all available. SJF picks P3 first (shortest at 2), then P4 (next shortest at 4), and finally P2. The starvation risk of SJF is clearly illustrated here: P2, despite having arrived early at $t=1$, is the last to finish because it has the longest burst time. Its waiting time of 8 units is the worst of all processes in this test case. As Hansen (2001) noted, this tendency to systematically postpone long jobs is the fundamental fairness problem of SJF.

C. Shortest Remaining Time (SRT) — Preemptive

How the Algorithm Works

Shortest Remaining Time is the preemptive counterpart of SJF. The scheduler continuously evaluates all available processes, and whenever a newly arrived process has a shorter remaining burst time than the currently executing one, the running process is preempted and returned to the ready queue. The new process with the shortest remaining time then takes over the CPU (Stallings, 2018).

In the simulation, SRT is implemented by advancing the clock one unit at a time, checking at each tick whether a newly arrived process should preempt the current one. This unit-step simulation ensures precise preemption



handling and accurate Gantt chart generation. Consecutive blocks belonging to the same process are then merged for readability.

Type: Preemptive

Advantages:

- Achieves the lowest possible average waiting time among preemptive algorithms under most workloads.
- Highly responsive — short processes inserted into the queue are served almost immediately.
- Efficiently utilizes CPU time by always favoring the process closest to completion.

Limitations:

- Frequent preemptions result in high context switching overhead, which can offset theoretical time savings (Arpaci-Dusseau & Arpaci-Dusseau, 2018).
- Starvation of long processes is a real and significant risk.
- Burst times must be known or accurately predicted for the algorithm to function correctly.



Figure 15. *Shortest Remaining Time Scheduling Algorithm*

Sample Test Case 1 — Shortest Remaining Time (SRT)

⚙️ Process Input

NUMBER OF PROCESSES

4

➕ Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P ₁	0	6	2
P ₂	1	3	1
P ₃	2	1	1
P ₄	5	4	4

⚙️ Run Simulation

Figure 16. *Process Input of SRT Case 1*



Figure 17. *Gantt Chart of SRT Case 1*

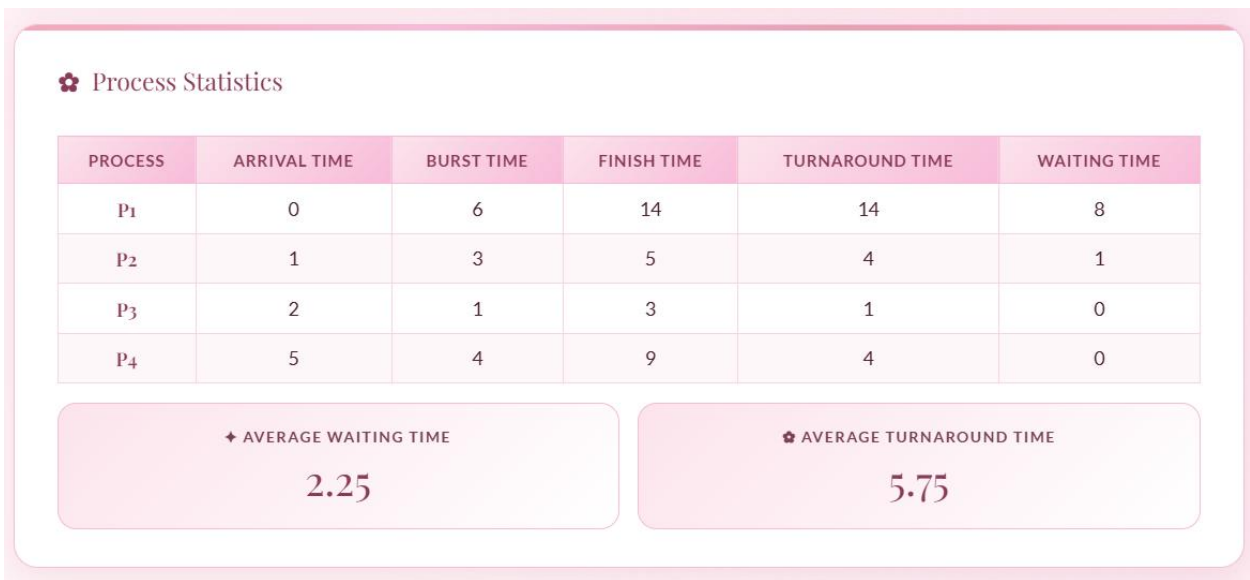


Figure 18. *Process Statistics of SRT Case 1*

This test case vividly illustrates how SRT handles multiple preemptions in rapid succession. P1 starts at $t=0$ with $BT=6$. At $t=1$, P2 arrives with remaining time $3 < P1$ remaining 5, so P1 is preempted. At $t=2$, P3 arrives with remaining time $1 < P2$ remaining 2, so P2 is preempted. P3 runs briefly and finishes at $t=3$. P2 then resumes with 2 units remaining and finishes at $t=5$. At $t=5$, P4 arrives with $BT=4 < P1$ remaining 5, so P4 runs before P1 resumes. The result is that P3 and P4 achieve zero waiting time, while P1 bears most of the delay at 8 units of waiting — a direct consequence of SRT's aggressive short-burst favoritism. The average waiting time of 2.25 is notably low, confirming SRT's theoretical strength for minimizing system-wide wait (Stallings, 2018).



Sample Test Case 2 — Shortest Remaining Time (SRT)

✿ Process Input

NUMBER OF PROCESSES

3

✦ Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	5	3
P2	2	2	1
P3	4	4	2

✦ Run Simulation

Figure 19. Process Input of SRT Case 2



Figure 20. Gantt Chart of SRT Case 2

✿ Process Statistics

PROCESS	ARRIVAL TIME	BURST TIME	FINISH TIME	TURNAROUND TIME	WAITING TIME
P1	0	5	7	7	2
P2	2	2	4	2	0
P3	4	4	11	7	3

✦ AVERAGE WAITING TIME

1.67

✦ AVERAGE TURNAROUND TIME

5.33

Figure 21. Process Statistics of SRT Case 2

With only three processes, this test case produces the best average waiting time of all SRT tests at 1.67. P1 starts at t=0. At t=2, P2 arrives with remaining time 2 < P1 remaining 3, triggering preemption. P2 finishes quickly at t=4. P1 resumes with 3 units remaining and runs until t=7. P3 arrives at t=4 but must wait for P1 to complete since



P1's remaining time of 3 is less than P3's full burst of 4. P3 then runs uninterrupted from $t=7$ to $t=11$. This case demonstrates that when new arrivals do not have shorter remaining times than the current process, SRT behaves like SJF with no unnecessary preemptions, resulting in clean execution and minimal overhead (Arpaci-Dusseau & Arpaci-Dusseau, 2018).

D. Round Robin (RR)

How the Algorithm Works

Round Robin assigns each process in the ready queue a fixed unit of CPU time called a time quantum. Processes are served in circular order: the first process runs for up to one quantum, then is placed at the back of the queue, and the next process takes over. If a process finishes before its quantum expires, the CPU is immediately released for the next process (Silberschatz et al., 2018).

In the simulation, the user sets the time quantum before running the algorithm. The program maintains a circular queue, enqueueing newly arrived processes after each quantum and correctly handling processes that finish within their allocated slice.

Type: Preemptive Requires: Time Quantum (user-defined)

Advantages:

- Provides the fairest distribution of CPU time among all processes.
- Ensures no process is indefinitely starved.
- Well-suited for time-sharing systems and interactive environments where responsiveness is critical.

Limitations:

- A small-time quantum leads to excessive context switching, increasing overhead and reducing effective throughput (Arpaci-Dusseau & Arpaci-Dusseau, 2018).
- A large time quantum causes Round Robin to degrade into FCFS behavior.
- Average Turnaround Time is often higher than that of SJF or SRT.



Figure 22. Round Robin Quantum 2 Scheduling Algorithm



Sample Test Case 1 — Round Robin (RR) | Quantum = 2

✿ Process Input

NUMBER OF PROCESSES

4

✦ Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	5	3
P2	1	4	1
P3	2	2	2
P4	3	6	4

✦ Run Simulation

Figure 23. Process Input of RR Case 1

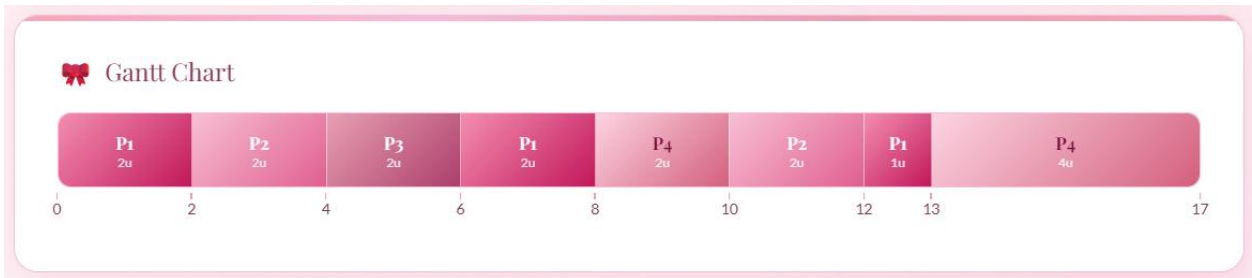


Figure 24. Gantt Chart of RR Case 1

✿ Process Statistics

PROCESS	ARRIVAL TIME	BURST TIME	FINISH TIME	TURNAROUND TIME	WAITING TIME
P1	0	5	13	13	8
P2	1	4	12	11	7
P3	2	2	6	4	2
P4	3	6	17	14	8

✦ AVERAGE WAITING TIME

6.25

✦ AVERAGE TURNAROUND TIME

10.50

Figure 25. Process Statistics of RR Case 1



With a quantum of 2, each process gets a turn in rapid rotation. P1 runs from $t=0$ to $t=2$, then P2 from $t=2$ to $t=4$, then P3 from $t=4$ to $t=6$ (P3 finishes exactly at $t=6$ since its burst is 2). The cycling continues as P4, P1, P2, and P4 take turns in subsequent rounds. P3 benefits the most from Round Robin because its short burst of 2 units is served in exactly one quantum, resulting in a turnaround time of just 4 and a waiting time of only 2. P4, however, with a burst of 6 units, requires three separate quanta spread across the timeline, leading to the worst turnaround time of 14. The high average waiting time of 6.25 reflects the inherent trade-off of Round Robin: fairness is achieved at the cost of efficiency, particularly for longer processes (Silberschatz et al., 2018).

Choose Scheduling Algorithm

FCFS
First-Come First-Served

SJF
Shortest Job First

SRT
Shortest Remaining Time

Round Robin
RR

Priority NP
Non-Preemptive

Priority P
Preemptive

Priority + RR
With Quantum

TIME QUANTUM

3

Figure 26. Round Robin Quantum 3 Scheduling Algorithm

Sample Test Case 2 — Round Robin (RR) | Quantum = 3

Process Input

NUMBER OF PROCESSES

3

Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	4	2
P2	0	7	1
P3	0	3	3

Run Simulation

Figure 27. Process Input of RR Case 2

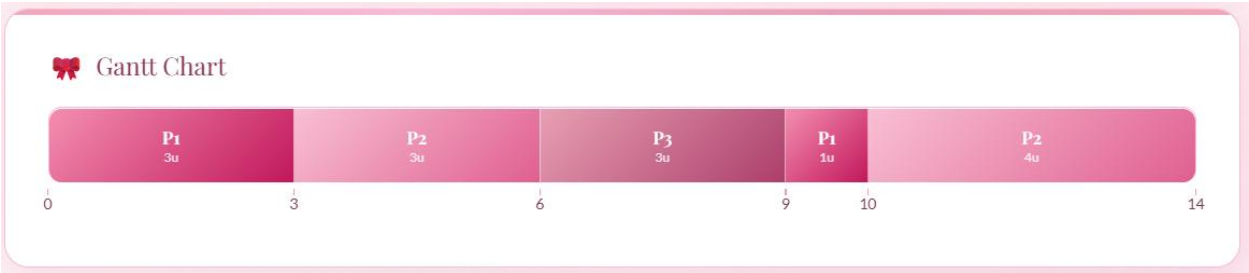


Figure 28. Gantt Chart of RR Case 2

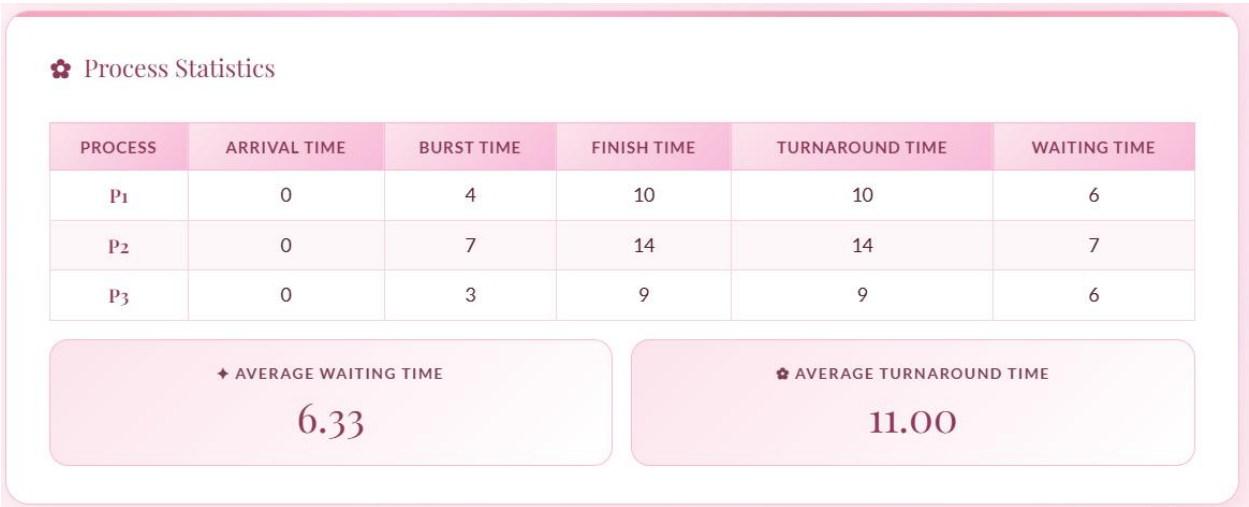


Figure 29. Process Statistics of RR Case 2

When all three processes arrive simultaneously at $t=0$, Round Robin distributes CPU time in strict rotation. Each receives a quantum of 3: P1 runs $t=0$ to $t=3$, P2 from $t=3$ to $t=6$, P3 from $t=6$ to $t=9$ (finishes exactly). In the second round, P1 has 1 unit remaining (finishes at $t=10$), then P2 resumes for its final 4 units and finishes at $t=14$. Comparing this to Test Case 1, the larger quantum of 3 reduces the total number of context switches — P3 and P1 each complete within two turns rather than being fragmented further — but the average metrics remain high because all processes start simultaneously and must wait through each other's full quantum each cycle. This test case reinforces Arpaci-Dusseau and Arpaci-Dusseau's (2018) observation that while Round Robin excels at fairness, it consistently trades higher average turnaround times for that fairness guarantee.

E. Priority Scheduling — Non-Preemptive

How the Algorithm Works

Each process is assigned a numeric priority value at the time of input. When the CPU becomes free, the scheduler selects the process in the ready queue with the highest priority — defined in this simulation as the lowest numeric value. The selected process then runs to completion without interruption. Arrival time serves as the tiebreaker when two processes share the same priority value (Silberschatz et al., 2018).



In the simulation, priority values are entered alongside arrival and burst times for all algorithms. For FCFS, SJF, SRT, and Round Robin, priority values are recorded but do not influence the scheduling decision. For Priority Scheduling, they become the primary criterion.

Type: Non-Preemptive Priority Convention: Lower numeric value = Higher priority

Advantages:

- Allows critical or time-sensitive processes to be served ahead of less important ones.
- Provides a straightforward mechanism for differentiating process importance.
- Non-preemptive variant is simple to implement and has predictable behavior.

Limitations:

- Low-priority processes may starve indefinitely if higher-priority processes continuously arrive (Hansen, 2001).
- Priority assignment is often subjective and may not reflect actual execution urgency.
- Without an aging mechanism, starvation cannot be prevented by the algorithm alone.

Choose Scheduling Algorithm

FCFS
First-Come First-Served

SJF
Shortest Job First

SRT
Shortest Remaining Time

Round Robin
RR

Priority NP
Non-Preemptive

Priority P
Preemptive

Priority + RR
With Quantum

PRIORITY DIRECTION

Lower value = Higher priority

Figure 30. Priority Non-Preemptive Scheduling Algorithm

Sample Test Case 1 — Priority Non-Preemptive

Process Input

NUMBER OF PROCESSES

4

Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	5	3
P2	1	3	1
P3	2	4	4
P4	3	2	2

Run Simulation

Figure 31. Process Input of Priority NP Case 1



Figure 32. Gantt Chart of Priority NP Case 1

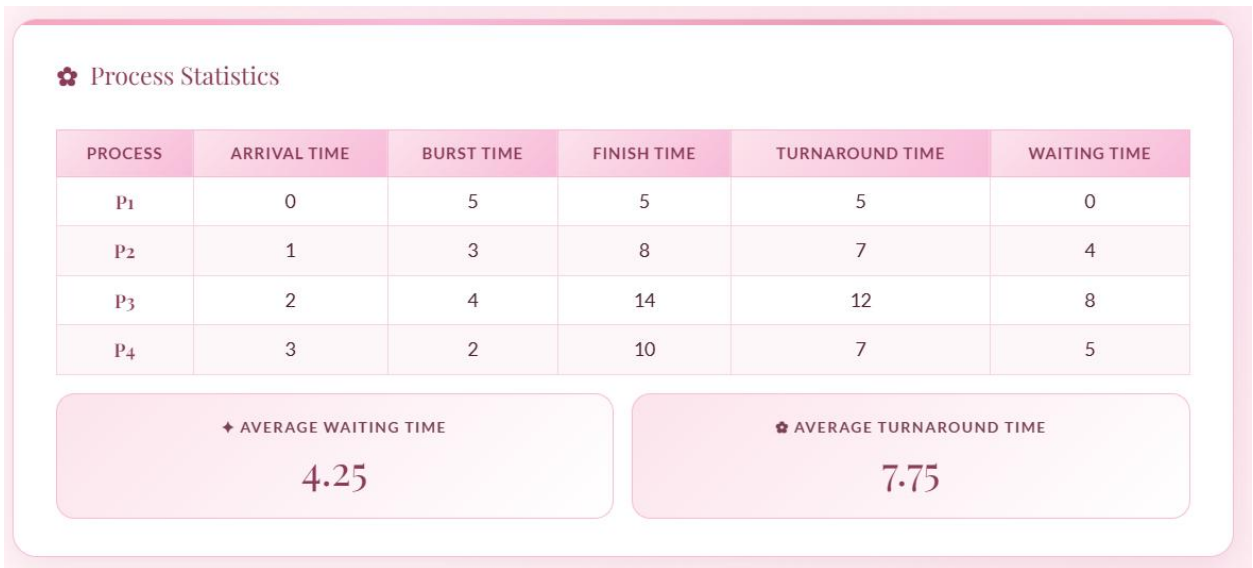


Figure 33. Process Statistics of Priority NP Case 1

At $t=0$, only P1 is available so it runs regardless of its priority of 3. Once P1 finishes at $t=5$, all remaining processes have arrived: P2 with priority 1 (highest), P4 with priority 2, and P3 with priority 4 (lowest). The scheduler correctly selects P2 first, then P4, and finally P3. This ordering demonstrates the primary strength of Priority Scheduling: important processes, as designated by their priority values, receive CPU access ahead of less critical ones regardless of burst time. However, P3 suffers the worst waiting time of 8 units purely because it was assigned the lowest priority, not because of any property of its actual work requirement. This illustrates that priority assignment carries significant responsibility — a misjudged priority value can impose severe delays on a process (Silberschatz et al., 2018).



Sample Test Case 2 — Priority Non-Preemptive

✿ Process Input

NUMBER OF PROCESSES

4

✦ Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	3	2
P2	1	5	4
P3	2	1	1
P4	4	4	3

✿ Run Simulation

Figure 34. Process Input of Priority NP Case 2



Figure 35. Process Input of Priority NP Case 2

✿ Process Statistics

PROCESS	ARRIVAL TIME	BURST TIME	FINISH TIME	TURNAROUND TIME	WAITING TIME
P1	0	3	3	3	0
P2	1	5	13	12	7
P3	2	1	4	2	1
P4	4	4	8	4	0

✦ AVERAGE WAITING TIME

2.00

✿ AVERAGE TURNAROUND TIME

5.25

Figure 36. Process Statistics of Priority NP Case 2



This test case achieves the best average waiting time among all Priority NP tests at 2.00. P1 runs first at $t=0$ due to sole availability. At $t=3$, P2 ($pr=4$) and P3 ($pr=1$) have arrived; the scheduler selects P3 because its priority of 1 is the highest. P3 finishes quickly at $t=4$, and P4 arrives just in time. At $t=4$, P2 ($pr=4$) and P4 ($pr=3$) are available; P4 is chosen. After P4 finishes at $t=8$, P2 finally executes. The outcome sharply demonstrates the starvation risk: P2 arrived at $t=1$ but does not finish until $t=13$, accumulating a waiting time of 7 units. Every process that arrived after P2 but with a higher priority was served before it. Without an aging mechanism that gradually elevates the priority of waiting processes, P2 could be delayed indefinitely in a continuous-arrival scenario (Hansen, 2001).

F. Priority Scheduling with Round Robin (PRR)

How the Algorithm Works

Priority Scheduling with Round Robin is a hybrid algorithm that combines the priority-based ordering of Priority Scheduling with the time-sliced fairness of Round Robin. Processes are grouped by priority level, and the highest-priority group is served first. Within each priority group, processes are scheduled in Round Robin fashion using a user-defined time quantum. When all processes in the highest-priority group are complete, the scheduler moves to the next priority level (Stallings, 2018).

In the simulation, the user configures both the time quantum and the priority direction. The program sorts the ready queue by priority at each scheduling decision, then applies Round Robin rotation among processes sharing the top priority. This approach closely mirrors the behavior of Multilevel Queue scheduling found in modern operating systems.

Type: Preemptive **Requires:** Time Quantum (user-defined) **Priority Convention:** Lower numeric value = Higher priority

Advantages:

- Prevents complete starvation of equal-priority processes by ensuring each receives a time quantum in rotation.
- Combines the urgency-sensitivity of priority scheduling with the fairness guarantees of Round Robin.
- More closely models the behavior of real-world OS schedulers than any single-criterion algorithm (Tanenbaum & Bos, 2015).

Limitations:

- More complex to implement correctly, particularly when processes of different priorities arrive dynamically.
- Performance remains sensitive to quantum size.
- Lower-priority processes can still experience significant waiting if higher-priority processes are long-running or continuously arriving.



Choose Scheduling Algorithm

FCFS
First-Come First-Served

SJF
Shortest Job First

SRT
Shortest Remaining Time

Round Robin
RR

Priority NP
Non-Preemptive

Priority P
Preemptive

Priority + RR
With Quantum

TIME QUANTUM
2

PRIORITY DIRECTION
Lower value = Higher priority

Figure 37. Priority Round Robin Scheduling Algorithm

Sample Test Case 1 — Priority Round Robin (PRR) | Quantum = 2

Process Input

NUMBER OF PROCESSES
4

Generate Table

PROCESS	ARRIVAL TIME	BURST TIME	PRIORITY
P1	0	6	2
P2	0	4	1
P3	0	5	1
P4	0	3	3

Run Simulation

Figure 38. Process Input of Priority Round Robin Case 1

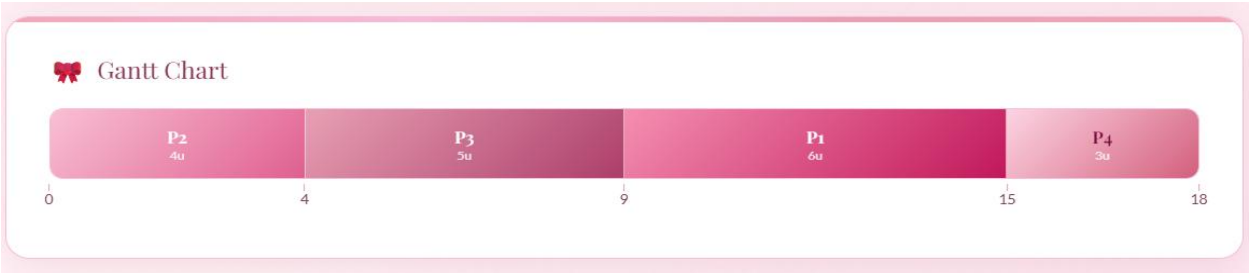


Figure 39. Gantt Chart of Priority Round Robin Case 1

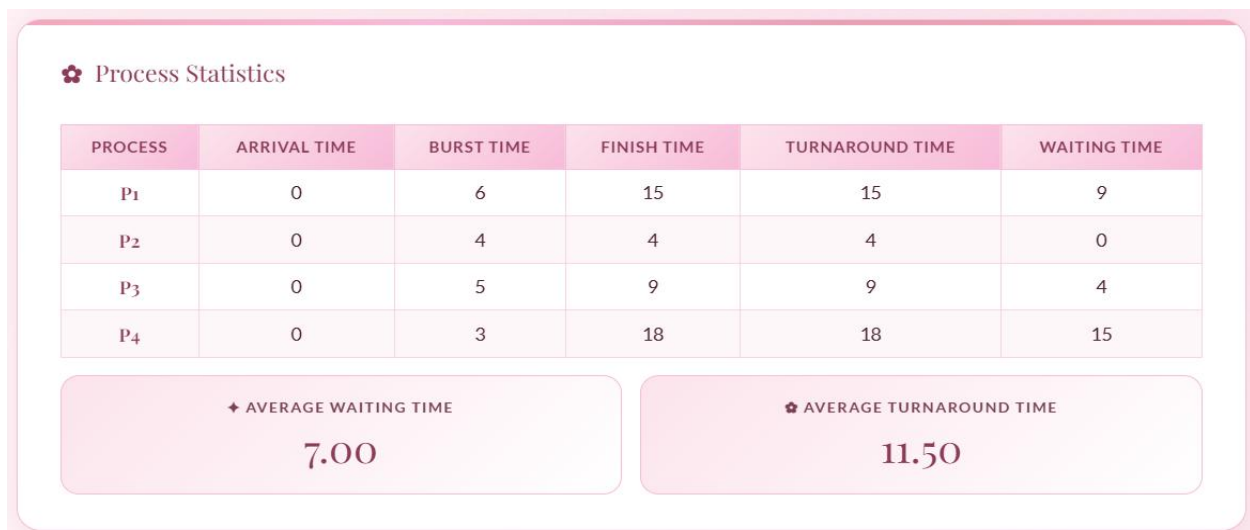


Figure 40. Process Statistics of Priority Round Robin Case 1

All four processes arrive simultaneously at $t=0$. P2 and P3 share the highest priority level of 1, so they are served first via Round Robin rotation with a quantum of 2. P2 runs from $t=0$ to $t=4$ and finishes. P3 runs from $t=4$ to $t=9$ and finishes. Once all priority-1 processes are complete, the scheduler moves to priority-2: P1 runs from $t=9$ to $t=15$. Finally, P4, with the lowest priority of 3, runs last from $t=15$ to $t=18$, accumulating the longest waiting time of 15 units. This test case clearly demonstrates both the strength and limitation of PRR: equal-priority processes are served fairly among themselves via Round Robin, but lower-priority processes may wait a very long time if higher-priority processes have significant total burst (Stallings, 2018).

Sample Test Case 2 — Priority Round Robin (PRR) | Quantum = 2



Figure 41. Process Input of Priority Round Robin Case 2



Figure 42. Gantt Chart of Priority Round Robin Case 2

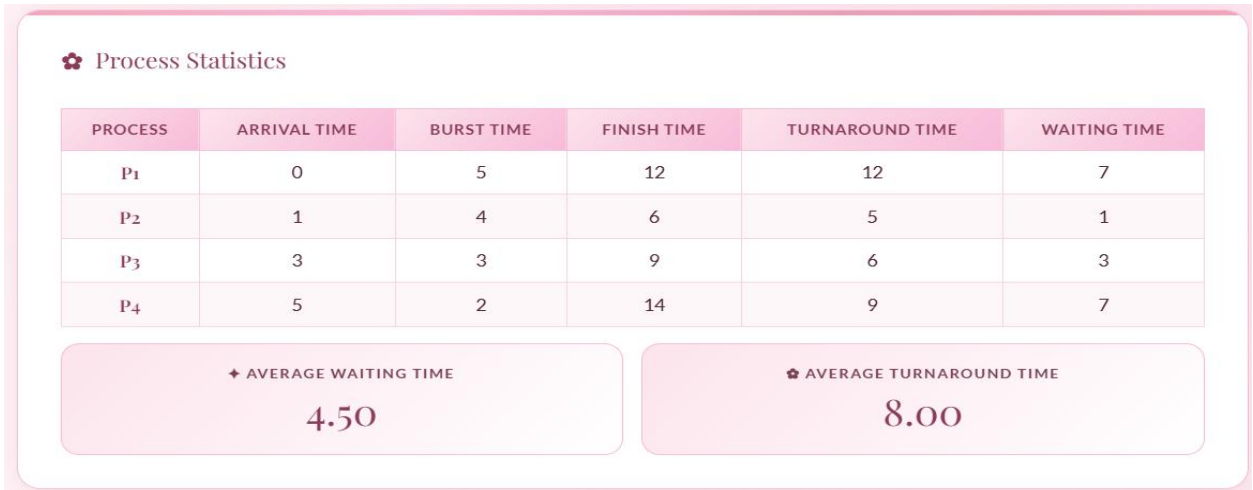


Figure 43. Process Statistics of Priority Round Robin Case 2

Unlike Test Case 1 where all processes arrived simultaneously, this test case involves staggered arrivals which adds scheduling complexity. P1 starts at $t=0$ as the only available process. At $t=1$, P2 arrives with higher priority 1, preempting P1 after its first quantum. P2 runs from $t=2$ to $t=6$ and finishes. P3 arrives at $t=3$ with the same priority 1 as P2 and joins the queue; it runs from $t=6$ to $t=9$ and finishes. With all priority-1 processes now complete, P1 resumes with 3 remaining units, running from $t=9$ to $t=12$. P4 arrives at $t=5$ with the lowest priority 3 and runs last from $t=12$ to $t=14$. The average waiting time of 4.50 is lower than Test Case 1 because the staggered arrivals prevent all high-priority work from blocking low-priority processes as severely from the start (Tanenbaum & Bos, 2015).

G. Comparative Summary of All Algorithms

The table below consolidates the average performance metrics from all test cases conducted in this simulation, providing a unified basis for comparing each algorithm's behavior across varied workloads.

Table 1. Summary of All Algorithms

Algorithm	Type	TC1 Avg. WT	TC1 Avg. TAT	TC2 Avg. WT	TC2 Avg. TAT	Overall Avg. WT	Overall Avg. TAT	Starvation Risk	Fairness
FCFS	Non-Preemptive	5.00	8.75	3.33	8.33	4.17	8.54	None	Moderate
SJF	Non-Preemptive	4.00	8.00	2.75	6.50	3.38	7.25	High	Low



SRT	Preemptive	2.25	5.75	1.67	5.33	1.96	5.54	High	Low
Round Robin	Preemptive	6.25	10.50	6.33	11.00	6.29	10.75	None	High
Priority NP	Non-Preemptive	4.25	7.75	2.00	5.25	3.13	6.50	High	Low
Priority + RR	Preemptive	7.00	11.50	4.50	8.00	5.75	9.75	Low	High

Analysis:

SRT delivered the lowest average waiting time across both test cases, with overall figures of 1.96 (AWT) and 5.54 (ATAT). This confirms its theoretical status as the preemptive algorithm that minimizes waiting time most aggressively (Arpaci-Dusseau & Arpaci-Dusseau, 2018). However, this performance came with the highest observed burden on individual long-running processes — in SRT Test Case 1, P1 accumulated a waiting time of 8 units while short-burst processes P3 and P4 experienced zero wait. The efficiency gain is real, but it is unevenly distributed.

SJF performed second-best overall in average waiting time at 3.38, confirming its theoretical optimality among non-preemptive algorithms. However, like SRT, it imposed starvation risk on long processes: in SJF Test Case 2, P2 waited 8 units while P3 and P4 each waited only 1–2 units.

FCFS produced consistent but suboptimal results, with average waiting times of 5.00 and 3.33 across its test cases. The difference between these two cases illustrates how strongly FCFS performance depends on the arrival order and relative burst lengths of processes — when a long process arrives first, as in Test Case 1, average waiting time increases significantly due to the convoy effect (Tanenbaum & Bos, 2015).

Priority Scheduling (Non-Preemptive) achieved an overall average waiting time of 3.13, competitive with SJF but structured around process importance rather than burst length. Its Test Case 2 delivered the best Priority NP result at only 2.00 AWT, though this was accompanied by P2's starvation waiting time of 7 units. Without an aging mechanism, the algorithm has no safeguard against this outcome.

Round Robin produced the highest average waiting times across both test cases (6.25 and 6.33), confirming that it trades efficiency for fairness. However, it is the only algorithm that completely eliminates starvation risk, and in both test cases, the waiting times across individual processes were the most evenly distributed — no single process was left severely disadvantaged. For interactive or time-sharing environments, this property is more valuable than raw average performance (Silberschatz et al., 2018).

Priority Scheduling with Round Robin showed the widest performance gap between its two test cases (7.00 vs. 4.50 AWT), reflecting its sensitivity to process arrival patterns. When all processes arrive simultaneously (Test Case 1), lower-priority processes are blocked entirely until higher-priority work completes, resulting in P4's worst-case waiting time of 15 units. When arrivals are staggered (Test Case 2), this blocking effect is moderated, producing



considerably better overall metrics. This variability highlights both the power and the contextual dependency of the hybrid approach — it is the most balanced algorithm when workloads arrive with temporal spread and closely mirrors the behavior of Multilevel Feedback Queue schedulers used in production operating systems (Stallings, 2018).



III. Summary

Key Insights Gained

Implementing all six scheduling algorithms within a single interactive simulation yielded insights that textbook study alone cannot fully convey. Most significantly, the exercise made concrete the fundamental tension at the heart of CPU scheduling: efficiency and fairness are competing objectives, and every algorithm represents a deliberate choice about which to prioritize. Algorithms that minimize average waiting time, such as SJF and SRT, do so by systematically favoring shorter or higher-priority processes — an approach that is efficient in aggregate but can be deeply unfair to individual long-running processes (Silberschatz et al., 2018). Conversely, algorithms that ensure fairness, such as Round Robin, accept higher average waiting times as the price of equitable resource distribution.

The simulation data collected across all twelve test cases reinforced this pattern consistently. SRT's overall average waiting time of 1.96 was the best of any algorithm tested, yet individual processes in SRT Test Case 1 experienced waiting times ranging from 0 to 8 units — a spread of 8 units between the best-served and worst-served process. Round Robin's overall average of 6.29 was the worst, yet its intra-test-case spread was consistently the narrowest, with no single process experiencing dramatically worse treatment than others. This numerical evidence gives concrete meaning to what operating systems literature describes abstractly as the efficiency-fairness trade-off.

A second key insight concerns the outsized influence of input parameters — particularly arrival timing — on algorithm behavior. Priority Scheduling with Round Robin's average waiting time dropped from 7.00 in Test Case 1 (simultaneous arrivals) to 4.50 in Test Case 2 (staggered arrivals), a difference of 2.50 units driven entirely by arrival pattern rather than any change in the algorithm itself. This reinforces Arpaci-Dusseau and Arpaci-Dusseau's (2018) observation that workload characteristics are as important as policy design in determining scheduling outcomes, and that no algorithm should be evaluated in isolation from the characteristics of the processes it will serve.

Practical Relevance of CPU Scheduling

The algorithms explored in this case study are not merely historical artifacts — they are the building blocks of the schedulers running in the operating systems used every day. Linux's Completely Fair Scheduler (CFS), for example, draws conceptually from both Round Robin and priority-based scheduling, using a red-black tree structure to track process runtimes and ensure proportional CPU allocation (Tanenbaum & Bos, 2015). Windows employs a priority-based preemptive scheduler with 32 priority levels, incorporating elements of both Priority Scheduling and Round Robin at each level (Stallings, 2018). Understanding the basic algorithms studied here provides the conceptual foundation necessary to reason about how these more sophisticated schedulers work, where they succeed, and where they may be tuned or improved.



As Hansen (2001) noted in tracing the evolution of operating systems, the progression from simple FCFS-like batch schedulers to modern multi-level feedback queues reflects decades of refinement driven by the practical experience of running real workloads on real hardware. The lessons encoded in that history — about the costs of starvation, the importance of responsiveness, and the difficulty of predicting burst times — remain directly relevant to systems design today.

Challenges Encountered and How They Were Addressed

The most technically demanding challenge in building the simulation was implementing preemptive scheduling correctly. In non-preemptive algorithms, the simulation advances in whole-process steps, which is straightforward to implement and reason about. Preemptive algorithms — particularly SRT and Preemptive Priority Scheduling — require the simulation clock to advance one unit at a time, checking at every tick whether a newly arrived process should displace the currently running one. Ensuring that the Gantt chart accurately captured every preemption, and that finish times, turnaround times, and waiting times were computed correctly even after multiple interruptions, required careful data structure design and thorough testing against manually computed expected outputs.

A second challenge was implementing Priority Scheduling with Round Robin as a genuinely hybrid algorithm rather than simply an alternate labeling of existing logic. Correctly interleaving processes of equal priority in a Round Robin rotation while still respecting the global priority ordering demanded additional queue management logic. This was addressed by sorting the ready queue by priority at each scheduling step and applying time-sliced rotation only among processes sharing the current highest-priority level.

Finally, designing the Gantt chart rendering to accurately reflect idle CPU periods — rather than silently skipping them — required careful handling of gaps between the completion of one process and the arrival of the next. These idle blocks were rendered explicitly in the simulation output to ensure that the timeline remained accurate and readable regardless of process arrival patterns.



IV. Source Code

Code Structure Overview

The program is organized into three main layers that work together to produce the simulation:

HTML (Structure) defines the user interface elements: the algorithm selector buttons, the process input table, the Run Simulation button, the Gantt chart container, and the statistics output area. All interactive elements are given unique IDs so that JavaScript can reference and manipulate them dynamically.

CSS (Presentation) applies the pastel pink coquette visual theme through CSS custom properties (variables) defined in: root. These variables — --rose, --blush, --petal, --wine, and others — are reused throughout the stylesheet to maintain visual consistency. Styles cover layout, color, typography, button states, table formatting, Gantt block coloring, and responsive behavior for smaller screens.

JavaScript (Logic) contains all scheduling algorithm implementations, user input handling, Gantt chart rendering, and statistics computation. The logic is organized into clearly separated sections: state management, algorithm functions, Gantt rendering, statistics rendering, and initialization.

JavaScript — Algorithm Selector and Table Management

```
<script>
// =====
// STATE
// =====
let selectedAlgo = 'FCFS';

// — Algorithm selector
document.querySelectorAll('.algo-btn').forEach(btn => {
  btn.addEventListener('click', () => {
    document.querySelectorAll('.algo-btn').forEach(b => b.classList.remove('active'));
    btn.classList.add('active');
    selectedAlgo = btn.dataset.algo;
    document.getElementById('quantum-row').style.display =
      (selectedAlgo === 'RR' || selectedAlgo === 'PRR') ? 'flex' : 'none';
    document.getElementById('priority-dir-row').style.display =
      (selectedAlgo === 'PNP' || selectedAlgo === 'PP' || selectedAlgo === 'PRR') ?
'flex' : 'none';
    refreshTable();
  });
});
```

Figure 44. Algorithm selector updates selectedAlgo and shows quantum/priority fields

The algorithm selector listens for click events on each button, updates the active state, stores the selected algorithm in selectedAlgo, and conditionally shows or hides the Time Quantum and Priority Direction fields. It then calls refreshTable() so that the process input table updates immediately.

```
function readCurrentValues() {
  const n = parseInt(document.getElementById('num-proc').value) || 0;
  const vals = [];
  for (let i = 0; i < n; i++) {
    vals.push({
      at: document.getElementById(`at_${i}`)?.value ?? i,
      bt: document.getElementById(`bt_${i}`)?.value ??
(Math.floor(Math.random()*7)+2),
      pr: document.getElementById(`pr_${i}`)?.value ?? (i+1),
    });
  }
  return vals;
}
```

Figure 45. *readCurrentValues* preserves process values before table rebuild

Before rebuilding the table HTML, this function saves all currently entered values, so they are not lost when the table is re-rendered after an algorithm switch.

```
function refreshTable(forceNew = false) {
  const n = parseInt(document.getElementById('num-proc').value);
  if (n < 3 || n > 10) { showToast('Please enter between 3 and 10 processes ✿');
  return; }
  const existing = forceNew ? [] : readCurrentValues();
  let html = `<table><thead><tr>
    <th>Process</th><th>Arrival Time</th><th>Burst Time</th><th>Priority</th>
  </tr></thead><tbody>`;
  for (let i = 0; i < n; i++) {
    const atVal = existing[i]?.at ?? i;
    const btVal = existing[i]?.bt ?? (Math.floor(Math.random()*7)+2);
    const prVal = existing[i]?.pr ?? (i+1);
    html += `<tr>
      <td style="font-family:'Playfair Display',serif;color:var(--wine);font-weight:600">P${i+1}</td>
      <td><input type="number" id="at_${i}" value="${atVal}" min="0"></td>
      <td><input type="number" id="bt_${i}" value="${btVal}" min="1"></td>
      <td><input type="number" id="pr_${i}" value="${prVal}" min="1"></td>
    </tr>`;
  }
  html += `</tbody></table>`;
  document.getElementById('process-table-wrap').innerHTML = html;
}
```

Figure 46. *refreshTable* generates input table with preserved/random values

The Priority column is always rendered for all algorithms. When *forceNew* is false (algorithm switch), existing values are preserved. When true (clicking Generate Table), fresh random values are assigned.

JavaScript — Scheduling Algorithms

```
function scheduleFCFS(procs) {
  const sorted = [...procs].sort((a,b) => a.at - b.at || a.idx - b.idx);
  const gantt = []; let t = 0;
```

```
for (const p of sorted) {  
  if (t < p.at) { gantt.push({id:'IDLE', start:t, end:p.at}); t = p.at; }  
  gantt.push({id:p.id, start:t, end:t+p.bt});  
  t += p.bt;  
}  
return gantt;  
}
```

Figure 47. *scheduleFCFS sorts by arrival*

FCFS iterates through processes in arrival order. If the current clock time is earlier than the next process's arrival, an IDLE block is inserted. Each process runs to completion before the next begins.

```
function scheduleSJF(procs) {  
  const rem = procs.map(p => ({...p, rem: p.bt}));  
  const done = new Set(); const gantt = []; let t = 0;  
  while (done.size < procs.length) {  
    const avail = rem.filter(p => !done.has(p.id) && p.at <= t);  
    if (!avail.length) { t++; continue; }  
    avail.sort((a,b) => a.bt - b.bt || a.at - b.at || a.idx - b.idx);  
    const p = avail[0]; done.add(p.id);  
    if (t < p.at) gantt.push({id:'IDLE', start:t, end:p.at});  
    gantt.push({id:p.id, start:t, end:t+p.bt}); t += p.bt;  
  }  
  return mergeGantt(gantt);  
}
```

Figure 48. *scheduleSJF selects shortest burst among available processes*

SJF filters available processes at each decision point and sorts them by burst time. The clock advances by 1 when no process is available, handling cases where the CPU must idle waiting for the next arrival.

```
function scheduleSRT(procs) {  
  const rem = procs.map(p => ({...p, rem: p.bt}));  
  const gantt = []; let t = 0;  
  let elapsed = 0; const total = procs.reduce((s,p)=>s+p.bt,0);  
  const maxT = procs.reduce((m,p)=>Math.max(m,p.at),0) + total + 5;  
  while (elapsed < total && t < maxT) {  
    const avail = rem.filter(p => p.rem > 0 && p.at <= t);  
    if (!avail.length) { gantt.push({id:'IDLE', start:t, end:t+1}); t++; continue; }  
    avail.sort((a,b) => a.rem - b.rem || a.at - b.at || a.idx - b.idx);  
    const p = avail[0];  
    gantt.push({id:p.id, start:t, end:t+1}); p.rem--; elapsed++; t++;  
  }  
  return mergeGantt(gantt);  
}
```

Figure 49. *scheduleSRT preempts on shorter remaining time arrivals*

SRT advances one time unit at a time, re-evaluating remaining burst times at every tick. This enables precise preemption — the moment a new process arrives with less remaining time than the current runner, it takes over. mergeGantt consolidates consecutive same-process entries for a cleaner chart.


```
function scheduleRR(procs, quantum) {
  const rem = procs.map(p => ({...p, rem:p.bt}));
  const gantt = []; let t = 0;
  const queue = []; const arrived = new Set();
  rem.filter(p=>p.at===0).forEach(p=>{queue.push(p);arrived.add(p.id)});
  let safety = 0;
  while (rem.some(p=>p.rem>0) && safety++ < 50000) {
    if (!queue.length) {
      const next = rem.filter(p=>p.rem>0&&!arrived.has(p.id)).sort((a,b)=>a.at-
b.at)[0];
      if (!next) break;
      gantt.push({id:'IDLE',start:t,end:next.at}); t=next.at;
      rem.filter(p=>p.at<=t&&p.rem>0&&!arrived.has(p.id)).forEach(p=>{queue.push(p);a
rrived.add(p.id)});
      continue;
    }
    const p = queue.shift();
    const slice = Math.min(quantum, p.rem);
    gantt.push({id:p.id, start:t, end:t+slice});
    t += slice; p.rem -= slice;
    rem.filter(p2=>p2.rem>0&&p2.at<=t&&!arrived.has(p2.id)).forEach(p2=>{queue.push(p
2);arrived.add(p2.id)});
    if (p.rem > 0) queue.push(p);
  }
  return mergeGantt(gantt);
}
```

Figure 50. *scheduleRR* uses circular queue for time quantum slices

The Round Robin implementation maintains an explicit queue and a Set to track which processes have arrived. After each quantum slice, newly arrived processes are enqueued before the current process is re-queued (if it still has remaining time), ensuring correct ordering.

```
function schedulePriorityNP(procs) {
  const rem = procs.map(p=>({...p})); const done=new Set(); const gantt=[]; let t=0;
  while (done.size < procs.length) {
    const avail = rem.filter(p=>!done.has(p.id)&&p.at<=t);
    if (!avail.length) { t++; continue; }
    avail.sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.at-b.at||a.idx-
b.idx);
    const p=avail[0]; done.add(p.id);
    if (t<p.at) gantt.push({id:'IDLE',start:t,end:p.at});
    gantt.push({id:p.id,start:t,end:t+p.bt}); t+=p.bt;
  }
  return mergeGantt(gantt);
}
```

Figure 51. *schedulePriorityNP* sorts by priority (non-preemptive)

normalPriority converts raw priority values into a comparable form based on the user's chosen convention. Negating the value when lower = higher priority allows the sort to work correctly in both directions without conditional branching in each algorithm.

```
function schedulePriorityP(procs) {
  const rem=procs.map(p=>({...p,rem:p.bt})); const gantt=[]; let t=0;
  let elapsed=0; const total=procs.reduce((s,p)=>s+p.bt,0);
  let safety=0;
  while (elapsed<total&&safety++<100000) {
    const avail=rem.filter(p=>p.rem>0&&p.at<=t);
    if (!avail.length){gantt.push({id:'IDLE',start:t,end:t+1});t++;continue;}
    avail.sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.at-b.at||a.idx-
b.idx);
    const p=avail[0];
    gantt.push({id:p.id,start:t,end:t+1}); p.rem--; elapsed++; t++;
  }
  return mergeGantt(gantt);
}
```

Figure 52. *schedulePriorityP* preempts based on highest priority

Preemptive Priority uses the same unit-step approach as SRT, but sorts by priority rather than remaining burst time. A newly arrived higher-priority process will displace the current runner at the next clock tick.

```
function schedulePRR(procs, quantum) {
  const rem=procs.map(p=>({...p,rem:p.bt}));
  const gantt=[]; let t=0; let safety=0;
  const queue=[]; const arrived=new Set();
  rem.filter(p=>p.at===0).sort((a,b)=>normalPriority(b.pr)-
normalPriority(a.pr)||a.idx-b.idx)
  .forEach(p=>{queue.push(p);arrived.add(p.id)});
  while (rem.some(p=>p.rem>0)&&safety++<100000) {
    if (!queue.length) {
      const nxt=rem.filter(p=>p.rem>0&&!arrived.has(p.id)).sort((a,b)=>a.at-b.at)[0];
      if (!nxt) break;
      gantt.push({id:'IDLE',start:t,end:nxt.at}); t=nxt.at;
      rem.filter(p=>p.rem>0&&p.at<=t&&!arrived.has(p.id))
        .sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.idx-b.idx)
        .forEach(p=>{queue.push(p);arrived.add(p.id)});
      continue;
    }
    queue.sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.idx-b.idx);
    const p=queue.shift();
    const slice=Math.min(quantum,p.rem);
    gantt.push({id:p.id,start:t,end:t+slice});
    t+=slice; p.rem-=slice;
    rem.filter(p2=>p2.rem>0&&p2.at<=t&&!arrived.has(p2.id))
      .forEach(p2=>{queue.push(p2);arrived.add(p2.id)});
    if (p.rem>0) queue.push(p);
  }
  return mergeGantt(gantt);
}
```



```
}
```

Figure 53. *schedulePRR rotates within top priority group only*

PRR combines the priority sort of Priority Scheduling with the slice-and-requeue mechanic of Round Robin. The queue is re-sorted by priority before each scheduling decision, ensuring higher-priority processes are always served first while equal-priority processes rotate fairly.

JavaScript — Statistics Computation

```
function computeStats(procs, gantt) {  
  const stats = {};  
  procs.forEach(p=>{stats[p.id]={id:p.id,at:p.at,bt:p.bt,ft:0,tat:0,wt:0}});  
  gantt.filter(g=>g.id!='IDLE').forEach(g=>{ stats[g.id].ft =  
Math.max(stats[g.id].ft, g.end); });  
  procs.forEach(p=>{  
    const s=stats[p.id];  
    s.tat = s.ft - s.at;  
    s.wt = s.tat - s.bt;  
  });  
  return Object.values(stats);  
}
```

Figure 54. *computeStats calculates TAT and WT from Gantt finish times*

Finish time is extracted from the Gantt array rather than computed separately, ensuring it is consistent with the visual output regardless of the algorithm. TAT and WT follow the standard OS formulas: $TAT = FT - AT$, $WT = TAT - BT$ (Silberschatz et al., 2018).

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<title>CPU Scheduling Simulator ❀</title>  
<link  
href="https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,400;0,600  
;1,400&family=Lato:wght@300;400;700&display=swap" rel="stylesheet">  
<style>  
  :root {  
    --rose: #f4a7b9;  
    --blush: #fce4ec;  
    --petal: #f8bbd9;  
    --dusty: #e8a0b4;  
    --cream: #fff5f8;  
    --wine: #8b3a5a;  
    --mauve: #c2687e;  
    --text: #4a2030;  
    --soft: #7a4055;  
    --white: #ffffff;  
    --shadow: rgba(180, 80, 110, 0.15);  
  }
```



```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: 'Lato', sans-serif;
  background: var(--cream);
  color: var(--text);
  min-height: 100vh;
  background-image:
    radial-gradient(circle at 10% 20%, rgba(244,167,185,0.3) 0%, transparent 40%),
    radial-gradient(circle at 90% 80%, rgba(248,187,217,0.25) 0%, transparent 40%),
    radial-gradient(circle at 50% 50%, rgba(252,228,236,0.5) 0%, transparent 70%);
}

.bow { display: flex; justify-content: center; padding: 18px 0 0; user-select: none; pointer-events: none; }
.bow svg { width: 80px; height: 40px; }

header { text-align: center; padding: 10px 20px 30px; }
header h1 {
  font-family: 'Playfair Display', serif;
  font-size: clamp(1.8rem, 4vw, 2.8rem);
  color: var(--wine);
  letter-spacing: 0.02em;
  line-height: 1.2;
}
header p { font-size: 0.88rem; color: var(--mauve); margin-top: 6px; font-style: italic; letter-spacing: 0.06em; }

.lace { text-align: center; color: var(--dusty); letter-spacing: 6px; font-size: 1rem; margin: 4px 0 18px; user-select: none; }

.container { max-width: 960px; margin: 0 auto; padding: 0 20px 60px; }

.card {
  background: var(--white);
  border: 1.5px solid var(--petal);
  border-radius: 18px;
  padding: 28px 32px;
  margin-bottom: 28px;
  box-shadow: 0 6px 28px var(--shadow);
  position: relative;
  overflow: hidden;
}
.card::before {
  content: '';
  position: absolute; top: 0; left: 0; right: 0; height: 4px;
  background: linear-gradient(90deg, var(--rose), var(--petal), var(--dusty), var(--rose));
}
.card-title {
```



```
font-family: 'Playfair Display', serif;
font-size: 1.15rem;
color: var(--wine);
margin-bottom: 18px;
display: flex; align-items: center; gap: 8px;
}
.card-title span { font-size: 1.1em; }

.form-row { display: flex; flex-wrap: wrap; gap: 16px; margin-bottom: 14px; }
.form-group { display: flex; flex-direction: column; flex: 1 1 140px; }
label { font-size: 0.78rem; color: var(--soft); font-weight: 700; letter-spacing:
0.08em; text-transform: uppercase; margin-bottom: 5px; }
input, select {
  border: 1.5px solid var(--petal);
  border-radius: 10px;
  padding: 9px 13px;
  font-family: 'Lato', sans-serif;
  font-size: 0.9rem;
  color: var(--text);
  background: var(--blush);
  outline: none;
  transition: border-color 0.2s, box-shadow 0.2s;
}
input:focus, select:focus {
  border-color: var(--rose);
  box-shadow: 0 0 3px rgba(244,167,185,0.25);
  background: var(--white);
}
select { cursor: pointer; }

#process-table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; font-size: 0.88rem; }
th {
  background: linear-gradient(135deg, var(--blush), var(--petal));
  color: var(--wine);
  font-weight: 700;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  font-size: 0.75rem;
  padding: 10px 12px;
  text-align: center;
  border: 1px solid var(--petal);
}
td { padding: 8px 12px; border: 1px solid #f9d4e2; text-align: center; vertical-
align: middle; }
tr:nth-child(even) td { background: rgba(252,228,236,0.35); }
tr:hover td { background: rgba(244,167,185,0.2); transition: 0.15s; }
td input { width: 72px; padding: 5px 8px; font-size: 0.85rem; background: var(--
white); border-radius: 7px; }

.btn {
  display: inline-flex; align-items: center; gap: 6px;
```



```
padding: 10px 22px;
border: none; border-radius: 50px; cursor: pointer;
font-family: 'Lato', sans-serif;
font-weight: 700; font-size: 0.85rem;
letter-spacing: 0.06em;
transition: transform 0.15s, box-shadow 0.2s;
}
.btn:hover { transform: translateY(-2px); box-shadow: 0 6px 18px var(--shadow); }
.btn:active { transform: translateY(0); }
.btn-primary { background: linear-gradient(135deg, var(--rose), var(--dusty));
color: var(--white); box-shadow: 0 4px 14px rgba(200,90,120,0.3); }
.btn-outline { background: var(--white); border: 1.5px solid var(--rose); color:
var(--wine); }
.btn-sm { padding: 6px 14px; font-size: 0.8rem; }

.algo-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(155px,
1fr)); gap: 10px; margin-bottom: 18px; }
.algo-btn {
padding: 10px 8px;
border: 1.5px solid var(--petal);
border-radius: 12px;
background: var(--blush);
color: var(--soft);
font-size: 0.8rem;
font-weight: 700;
letter-spacing: 0.04em;
text-align: center;
cursor: pointer;
transition: all 0.2s;
}
.algo-btn:hover { border-color: var(--rose); background: var(--white); }
.algo-btn.active {
background: linear-gradient(135deg, var(--rose), var(--dusty));
color: var(--white);
border-color: var(--dusty);
box-shadow: 0 4px 12px rgba(200,90,120,0.25);
}

#quantum-row { display: none; }
#priority-dir-row { display: none; }
#results { display: none; }

/* — Gantt Chart: no horizontal scroll — */
.gantt-outer { width: 100%; }

.gantt-chart {
display: flex;
align-items: stretch;
width: 100%;
height: 56px;
border-radius: 10px;
overflow: hidden;
```



```
border: 1.5px solid var(--petal);
}

.gantt-block {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  overflow: hidden;
  border-right: 1px solid rgba(255,255,255,0.25);
  transition: filter 0.15s;
  min-width: 0;
}
.gantt-block:last-child { border-right: none; }
.gantt-block:hover { filter: brightness(0.88); }

.gantt-block .proc-label {
  font-family: 'Playfair Display', serif;
  font-size: 0.82rem;
  font-weight: 600;
  color: #fff;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  max-width: 100%;
  padding: 0 3px;
}
.gantt-block .duration {
  font-size: 0.6rem;
  color: rgba(255,255,255,0.85);
  white-space: nowrap;
}

.gantt-idle { background: repeating-linear-gradient(45deg, #f0c4d4, #f0c4d4 4px,
#fce4ec 4px, #fce4ec 10px); }
.gantt-idle .proc-label { color: var(--soft); }
.gantt-idle .duration { color: var(--mauve); }

/* Time axis */
.gantt-axis {
  position: relative;
  width: 100%;
  height: 22px;
  margin-top: 4px;
  margin-bottom: 8px;
}
.gantt-tick {
  position: absolute;
  transform: translateX(-50%);
  font-size: 0.68rem;
  color: var(--soft);
  white-space: nowrap;
```




```
    line-height: 1;
}
.gantt-tick::before {
    content: '';
    display: block;
    width: 1px;
    height: 5px;
    background: var(--dusty);
    margin: 0 auto 2px;
}

/* Process colors */
.c0 { background: linear-gradient(135deg, #f48fb1, #c2185b); }
.c1 { background: linear-gradient(135deg, #f9c0d4, #e06090); }
.c2 { background: linear-gradient(135deg, #e8a0b4, #ad3f6b); }
.c3 { background: linear-gradient(135deg, #fdd5e3, #d4607e); }
.c3 .proc-label { color: #7a1040; }
.c4 { background: linear-gradient(135deg, #f06292, #880e4f); }
.c5 { background: linear-gradient(135deg, #fbbcd1, #c2527a); }
.c6 { background: linear-gradient(135deg, #f8d0df, #b5415f); }
.c6 .proc-label { color: #6a1535; }
.c7 { background: linear-gradient(135deg, #e57399, #7b1f40); }

.stats-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 14px; margin-top: 14px; }
.stat-card {
    background: linear-gradient(135deg, var(--blush), var(--white));
    border: 1px solid var(--petal);
    border-radius: 14px;
    padding: 16px 18px;
    text-align: center;
}
.stat-card .stat-label { font-size: 0.72rem; color: var(--soft); text-transform: uppercase; letter-spacing: 0.09em; font-weight: 700; }
.stat-card .stat-value { font-family: 'Playfair Display', serif; font-size: 1.9rem; color: var(--wine); margin-top: 4px; }

.results-table { width: 100%; border-collapse: collapse; font-size: 0.88rem; margin-top: 10px; }
.results-table th { background: linear-gradient(135deg, var(--blush), var(--petal)); color: var(--wine); font-weight: 700; letter-spacing: 0.06em; text-transform: uppercase; font-size: 0.75rem; padding: 10px; border: 1px solid var(--petal); text-align: center; }
.results-table td { padding: 9px 10px; border: 1px solid #f9d4e2; text-align: center; }
.results-table tr:nth-child(even) td { background: rgba(252,228,236,0.3); }

#toast {
    position: fixed; bottom: 28px; left: 50%; transform: translateX(-50%) translateY(80px);
    background: linear-gradient(135deg, var(--rose), var(--dusty));
    color: white; padding: 12px 28px; border-radius: 50px;
}
```




```
font-size: 0.88rem; font-weight: 700;
box-shadow: 0 6px 24px rgba(180,60,100,0.3);
transition: transform 0.35s cubic-bezier(.34,1.56,.64,1);
z-index: 999; white-space: nowrap;
}
#toast.show { transform: translateX(-50%) translateY(0); }

@media(max-width: 600px) {
  .card { padding: 20px 16px; }
  .stats-grid { grid-template-columns: 1fr; }
}

@keyframes fadeUp { from { opacity:0; transform:translateY(18px); } to { opacity:1;
transform:translateY(0); } }
.card { animation: fadeUp 0.4s ease both; }
.card:nth-child(2) { animation-delay: 0.07s; }
.card:nth-child(3) { animation-delay: 0.13s; }
.card:nth-child(4) { animation-delay: 0.19s; }
</style>
</head>
<body>

<div class="bow">
  <svg viewBox="0 0 120 60" fill="none" xmlns="http://www.w3.org/2000/svg">
    <ellipse cx="35" cy="30" rx="30" ry="18" fill="#f8bbd9" opacity="0.7"/>
    <ellipse cx="85" cy="30" rx="30" ry="18" fill="#f8bbd9" opacity="0.7"/>
    <ellipse cx="35" cy="30" rx="20" ry="12" fill="#f4a7b9" opacity="0.8"/>
    <ellipse cx="85" cy="30" rx="20" ry="12" fill="#f4a7b9" opacity="0.8"/>
    <line x1="50" y1="22" x2="70" y2="22" stroke="#e8a0b4" stroke-width="2"/>
    <line x1="50" y1="38" x2="70" y2="38" stroke="#e8a0b4" stroke-width="2"/>
    <circle cx="60" cy="30" r="8" fill="#fce4ec" stroke="#f4a7b9" stroke-
width="1.5"/>
    <circle cx="60" cy="30" r="5" fill="#f4a7b9"/>
    <path d="M54 36 Q48 50 42 54" stroke="#f8bbd9" stroke-width="2.5" stroke-
linecap="round" fill="none"/>
    <path d="M66 36 Q72 50 78 54" stroke="#f8bbd9" stroke-width="2.5" stroke-
linecap="round" fill="none"/>
  </svg>
</div>

<header>
  <h1>CPU Scheduling Simulator</h1>
  <p>🌸 &nbsp; Operating Systems Case Study &nbsp; 🌸</p>
</header>
<div class="lace">◆ ◆ ◆ ◆ ◆ ◆</div>

<div class="container">

  <!-- STEP 1: Algorithm -->
  <div class="card">
    <div class="card-title"><span>👉</span> Choose Scheduling Algorithm</div>
    <div class="algo-grid">
```



```
<div class="algo-btn active" data-algo="FCFS">FCFS<br><small style="font-  
weight:400;font-size:0.7rem">First-Come First-Served</small></div>  
<div class="algo-btn" data-algo="SJF">SJF<br><small style="font-  
weight:400;font-size:0.7rem">Shortest Job First</small></div>  
<div class="algo-btn" data-algo="SRT">SRT<br><small style="font-  
weight:400;font-size:0.7rem">Shortest Remaining Time</small></div>  
<div class="algo-btn" data-algo="RR">Round Robin<br><small style="font-  
weight:400;font-size:0.7rem">RR</small></div>  
<div class="algo-btn" data-algo="PNP">Priority NP<br><small style="font-  
weight:400;font-size:0.7rem">Non-Preemptive</small></div>  
<div class="algo-btn" data-algo="PP">Priority P<br><small style="font-  
weight:400;font-size:0.7rem">Preemptive</small></div>  
<div class="algo-btn" data-algo="PRR">Priority + RR<br><small style="font-  
weight:400;font-size:0.7rem">With Quantum</small></div>  
</div>  
<div class="form-row" id="quantum-row">  
  <div class="form-group" style="max-width:180px">  
    <label>Time Quantum</label>  
    <input type="number" id="quantum" value="2" min="1">  
  </div>  
</div>  
<div class="form-row" id="priority-dir-row">  
  <div class="form-group" style="max-width:260px">  
    <label>Priority Direction</label>  
    <select id="priority-dir">  
      <option value="lower">Lower value = Higher priority</option>  
      <option value="higher">Higher value = Higher priority</option>  
    </select>  
  </div>  
</div>  
</div>  
  
<!-- STEP 2: Process Input -->  
<div class="card">  
  <div class="card-title"><span>✿</span> Process Input</div>  
  <div class="form-row">  
    <div class="form-group" style="max-width:200px">  
      <label>Number of Processes</label>  
      <input type="number" id="num-proc" value="4" min="3" max="10">  
    </div>  
    <div style="display:flex;align-items:flex-end;padding-bottom:1px">  
      <button class="btn btn-outline btn-sm" onclick="generateTable()">◆ Generate  
Table</button>  
    </div>  
  </div>  
  <div id="process-table-wrap"></div>  
</div>  
  
<!-- STEP 3: Run -->  
<div class="card" style="text-align:center; padding: 24px;">  
  <button class="btn btn-primary" onclick="runScheduler()" style="font-size:1rem;  
padding:13px 38px;">
```



```

    ✿ &nbsp; Run Simulation
  </button>
</div>

<!-- RESULTS -->
<div id="results">

  <div class="card">
    <div class="card-title"><span>🐛</span> Gantt Chart</div>
    <div class="gantt-outer">
      <div class="gantt-chart" id="gantt-chart"></div>
      <div class="gantt-axis" id="gantt-axis"></div>
    </div>
  </div>

  <div class="card">
    <div class="card-title"><span>✿</span> Process Statistics</div>
    <div id="process-table-wrap-results" style="overflow-x:auto;"></div>
    <div class="stats-grid" id="avg-stats"></div>
  </div>

</div>

</div>

<div id="toast"></div>

<script>
// =====
// STATE
// =====
let selectedAlgo = 'FCFS';

// — Algorithm selector
document.querySelectorAll('.algo-btn').forEach(btn => {
  btn.addEventListener('click', () => {
    document.querySelectorAll('.algo-btn').forEach(b =>
b.classList.remove('active'));
    btn.classList.add('active');
    selectedAlgo = btn.dataset.algo;
    document.getElementById('quantum-row').style.display =
(selectedAlgo === 'RR' || selectedAlgo === 'PRR') ? 'flex' : 'none';
    document.getElementById('priority-dir-row').style.display =
(selectedAlgo === 'PNP' || selectedAlgo === 'PP' || selectedAlgo === 'PRR') ?
'flex' : 'none';
    refreshTable();
  });
});

// — Read current table values before rebuilding
function readCurrentValues() {
  const n = parseInt(document.getElementById('num-proc').value) || 0;

```

```
const vals = [];  
for (let i = 0; i < n; i++) {  
  vals.push({  
    at: document.getElementById(`at_${i}`)?.value ?? i,  
    bt: document.getElementById(`bt_${i}`)?.value ??  
(Math.floor(Math.random()*7)+2),  
    pr: document.getElementById(`pr_${i}`)?.value ?? (i+1),  
  });  
}  
return vals;  
}  
  
// — Refresh table (preserves existing AT/BT/PR values)  
function refreshTable(forceNew = false) {  
  const n = parseInt(document.getElementById('num-proc').value);  
  if (n < 3 || n > 10) { showToast('Please enter between 3 and 10 processes 🍀');  
return; }  
  const existing = forceNew ? [] : readCurrentValues();  
  let html = `<table><thead><tr>  
    <th>Process</th><th>Arrival Time</th><th>Burst Time</th><th>Priority</th>  
</tr></thead><tbody>`;  
  for (let i = 0; i < n; i++) {  
    const atVal = existing[i]?.at ?? i;  
    const btVal = existing[i]?.bt ?? (Math.floor(Math.random()*7)+2);  
    const prVal = existing[i]?.pr ?? (i+1);  
    html += `<tr>  
      <td style="font-family: 'Playfair Display', serif; color: var(--wine); font-weight: 600">P${i+1}</td>  
      <td><input type="number" id="at_${i}" value="${atVal}" min="0"></td>  
      <td><input type="number" id="bt_${i}" value="${btVal}" min="1"></td>  
      <td><input type="number" id="pr_${i}" value="${prVal}" min="1"></td>  
    </tr>`;  
  }  
  html += `</tbody></table>`;  
  document.getElementById('process-table-wrap').innerHTML = html;  
}  
  
// — Generate process table (fresh random values)  
function generateTable() {  
  const n = parseInt(document.getElementById('num-proc').value);  
  if (n < 3 || n > 10) { showToast('Please enter between 3 and 10 processes 🍀');  
return; }  
  refreshTable(true);  
}  
  
// — Toast notification  
function showToast(msg) {  
  const t = document.getElementById('toast');  
  t.textContent = msg;  
  t.classList.add('show');  
  setTimeout(() => t.classList.remove('show'), 2800);  
}
```



```
// =====  
// SCHEDULING ALGORITHMS  
// =====  
function getProcesses() {  
  const n = parseInt(document.getElementById('num-proc').value);  
  const needPriority = ['PNP','PP','PRR'].includes(selectedAlgo);  
  const procs = [];  
  for (let i = 0; i < n; i++) {  
    const at = parseInt(document.getElementById(`at_${i}`)?.value ?? 0);  
    const bt = parseInt(document.getElementById(`bt_${i}`)?.value ?? 1);  
    const pr = needPriority ? parseInt(document.getElementById(`pr_${i}`)?.value ??  
1) : 0;  
    procs.push({ id: `P${i+1}`, at, bt, pr, idx: i });  
  }  
  return procs;  
}  
  
function normalPriority(pr) {  
  const dir = document.getElementById('priority-dir').value;  
  return dir === 'lower' ? -pr : pr;  
}  
  
function scheduleFCFS(procs) {  
  const sorted = [...procs].sort((a,b) => a.at - b.at || a.idx - b.idx);  
  const gantt = []; let t = 0;  
  for (const p of sorted) {  
    if (t < p.at) { gantt.push({id:'IDLE', start:t, end:p.at}); t = p.at; }  
    gantt.push({id:p.id, start:t, end:t+p.bt});  
    t += p.bt;  
  }  
  return gantt;  
}  
  
function scheduleSJF(procs) {  
  const rem = procs.map(p => ({...p, rem: p.bt}));  
  const done = new Set(); const gantt = []; let t = 0;  
  while (done.size < procs.length) {  
    const avail = rem.filter(p => !done.has(p.id) && p.at <= t);  
    if (!avail.length) { t++; continue; }  
    avail.sort((a,b) => a.bt - b.bt || a.at - b.at || a.idx - b.idx);  
    const p = avail[0]; done.add(p.id);  
    if (t < p.at) gantt.push({id:'IDLE', start:t, end:p.at});  
    gantt.push({id:p.id, start:t, end:t+p.bt}); t += p.bt;  
  }  
  return mergeGantt(gantt);  
}  
  
function scheduleSRT(procs) {  
  const rem = procs.map(p => ({...p, rem: p.bt}));  
  const gantt = []; let t = 0;  
  let elapsed = 0; const total = procs.reduce((s,p)=>s+p.bt,0);
```

```
const maxT = procs.reduce((m,p)=>Math.max(m,p.at),0) + total + 5;
while (elapsed < total && t < maxT) {
  const avail = rem.filter(p => p.rem > 0 && p.at <= t);
  if (!avail.length) { gantt.push({id:'IDLE', start:t, end:t+1}); t++; continue; }
  avail.sort((a,b) => a.rem - b.rem || a.at - b.at || a.idx - b.idx);
  const p = avail[0];
  gantt.push({id:p.id, start:t, end:t+1}); p.rem--; elapsed++; t++;
}
return mergeGantt(gantt);
}

function scheduleRR(procs, quantum) {
  const rem = procs.map(p => ({...p, rem:p.bt}));
  const gantt = []; let t = 0;
  const queue = []; const arrived = new Set();
  rem.filter(p=>p.at===0).forEach(p=>{queue.push(p);arrived.add(p.id)});
  let safety = 0;
  while (rem.some(p=>p.rem>0) && safety++ < 50000) {
    if (!queue.length) {
      const next = rem.filter(p=>p.rem>0&&!arrived.has(p.id)).sort((a,b)=>a.at-
b.at)[0];
      if (!next) break;
      gantt.push({id:'IDLE',start:t,end:next.at}); t=next.at;
      rem.filter(p=>p.at<=t&&p.rem>0&&!arrived.has(p.id)).forEach(p=>{queue.push(p);a
rrived.add(p.id)});
      continue;
    }
    const p = queue.shift();
    const slice = Math.min(quantum, p.rem);
    gantt.push({id:p.id, start:t, end:t+slice});
    t += slice; p.rem -= slice;
    rem.filter(p2=>p2.rem>0&&p2.at<=t&&!arrived.has(p2.id)).forEach(p2=>{queue.push(p
2);arrived.add(p2.id)});
    if (p.rem > 0) queue.push(p);
  }
  return mergeGantt(gantt);
}

function schedulePriorityNP(procs) {
  const rem = procs.map(p=>({...p})); const done=new Set(); const gantt=[]; let t=0;
  while (done.size < procs.length) {
    const avail = rem.filter(p=>!done.has(p.id)&&p.at<=t);
    if (!avail.length) { t++; continue; }
    avail.sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.at-b.at||a.idx-
b.idx);
    const p=avail[0]; done.add(p.id);
    if (t<p.at) gantt.push({id:'IDLE',start:t,end:p.at});
    gantt.push({id:p.id,start:t,end:t+p.bt}); t+=p.bt;
  }
  return mergeGantt(gantt);
}
```

```
function schedulePriorityP(procs) {
  const rem=procs.map(p=>({...p,rem:p.bt})); const gantt=[]; let t=0;
  let elapsed=0; const total=procs.reduce((s,p)=>s+p.bt,0);
  let safety=0;
  while (elapsed<total&&safety++<100000) {
    const avail=rem.filter(p=>p.rem>0&&p.at<=t);
    if (!avail.length){gantt.push({id:'IDLE',start:t,end:t+1});t++;continue;}
    avail.sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.at-b.at||a.idx-
b.idx);
    const p=avail[0];
    gantt.push({id:p.id,start:t,end:t+1}); p.rem--; elapsed++; t++;
  }
  return mergeGantt(gantt);
}

function schedulePRR(procs, quantum) {
  const rem=procs.map(p=>({...p,rem:p.bt}));
  const gantt=[]; let t=0; let safety=0;
  const queue=[]; const arrived=new Set();
  rem.filter(p=>p.at===0).sort((a,b)=>normalPriority(b.pr)-
normalPriority(a.pr)||a.idx-b.idx)
    .forEach(p=>{queue.push(p);arrived.add(p.id);});
  while (rem.some(p=>p.rem>0)&&safety++<100000) {
    if (!queue.length) {
      const nxt=rem.filter(p=>p.rem>0&&!arrived.has(p.id)).sort((a,b)=>a.at-b.at)[0];
      if (!nxt) break;
      gantt.push({id:'IDLE',start:t,end:nxt.at}); t=nxt.at;
      rem.filter(p=>p.rem>0&&p.at<=t&&!arrived.has(p.id))
        .sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.idx-b.idx)
        .forEach(p=>{queue.push(p);arrived.add(p.id);});
      continue;
    }
    queue.sort((a,b)=>normalPriority(b.pr)-normalPriority(a.pr)||a.idx-b.idx);
    const p=queue.shift();
    const slice=Math.min(quantum,p.rem);
    gantt.push({id:p.id,start:t,end:t+slice});
    t+=slice; p.rem-=slice;
    rem.filter(p2=>p2.rem>0&&p2.at<=t&&!arrived.has(p2.id))
      .forEach(p2=>{queue.push(p2);arrived.add(p2.id);});
    if (p.rem>0) queue.push(p);
  }
  return mergeGantt(gantt);
}

function mergeGantt(gantt) {
  if (!gantt.length) return gantt;
  const merged=[{...gantt[0]}];
  for (let i=1;i<gantt.length;i++){
    const last=merged[merged.length-1];
    if (gantt[i].id===last.id&&gantt[i].start===last.end) last.end=gantt[i].end;
    else merged.push({...gantt[i]});
  }
}
```




```
    return merged;
}

// — Compute WT / TAT from gantt
function computeStats(procs, gantt) {
    const stats = {};
    procs.forEach(p=>{stats[p.id]={id:p.id,at:p.at,bt:p.bt,ft:0,tat:0,wt:0}});
    gantt.filter(g=>g.id!=='IDLE').forEach(g=>{ stats[g.id].ft =
Math.max(stats[g.id].ft, g.end); });
    procs.forEach(p=>{
        const s=stats[p.id];
        s.tat = s.ft - s.at;
        s.wt  = s.tat - s.bt;
    });
    return Object.values(stats);
}

// =====
// RUN
// =====

function runScheduler() {
    const procs = getProcesses();
    if (!procs.length) { showToast('Please generate the process table first ✿');
return; }
    for (const p of procs) {
        if (isNaN(p.at)||isNaN(p.bt)||p.bt<1) { showToast(`Invalid input for ${p.id}
✿`); return; }
    }
    const q = parseInt(document.getElementById('quantum').value)||2;
    let gantt;
    switch(selectedAlgo) {
        case 'FCFS': gantt=scheduleFCFS(procs); break;
        case 'SJF':  gantt=scheduleSJF(procs); break;
        case 'SRT':  gantt=scheduleSRT(procs); break;
        case 'RR':   gantt=scheduleRR(procs,q); break;
        case 'PNP':  gantt=schedulePriorityNP(procs); break;
        case 'PP':   gantt=schedulePriorityP(procs); break;
        case 'PRR':  gantt=schedulePRR(procs,q); break;
        default: gantt=[];
    }
    const stats = computeStats(procs, gantt);
    renderGantt(gantt, procs);
    renderStats(stats);
    document.getElementById('results').style.display='block';
    document.getElementById('results').scrollIntoView({behavior:'smooth',
block:'start'});
    showToast('Simulation complete! ✿');
}

function procColor(id, procs) {
    const idx = procs.findIndex(p=>p.id===id);
    return `c${idx % 8}`;
}
```

```
}

// — Render Gantt (no horizontal scroll)
function renderGantt(gantt, procs) {
  const total = gantt.reduce((m,g)=>Math.max(m,g.end),0);
  if (total === 0) return;

  let blocksHtml = '';
  gantt.forEach(g => {
    const span = g.end - g.start;
    const pct = (span / total * 100).toFixed(3);
    const cls = g.id === 'IDLE' ? 'gantt-idle' : procColor(g.id, procs);
    const label = g.id === 'IDLE' ? '-' : g.id;
    blocksHtml += `

48


```



```

</tr></thead><tbody>`;
stats.forEach(p=>{
  html+=`<tr>
    <td style="font-family:'Playfair Display',serif;color:var(--wine);font-
weight:600">${p.id}</td>
    <td>${p.at}</td><td>${p.bt}</td><td>${p.ft}</td>
    <td>${p.tat}</td><td>${p.wt}</td>
  </tr>`;
});
html+=`</tbody></table>`;
document.getElementById('process-table-wrap-results').innerHTML=html;

document.getElementById('avg-stats').innerHTML=`
  <div class="stat-card">
    <div class="stat-label">♦ Average Waiting Time</div>
    <div class="stat-value">${avgWT}</div>
  </div>
  <div class="stat-card">
    <div class="stat-label">✿ Average Turnaround Time</div>
    <div class="stat-value">${avgTAT}</div>
  </div>`;
}

// — Init
window.addEventListener('DOMContentLoaded', ()=>{ generateTable(); });
</script>
</body>
</html>

```

Figure 55. CPU Scheduling Source Code

This code creates an interactive **CPU Scheduling Simulator** with a soft, pastel-themed interface, designed to demonstrate how different process scheduling algorithms work in operating systems. Built with HTML, CSS, and JavaScript, it lets users select from methods like FCFS, SJF, SRT, Round Robin, and priority-based scheduling, input or generate process details such as arrival time, burst time, and priority, then run simulations to visualize the execution flow and performance metrics. The styling uses custom color variables, elegant typography from external font sources, and responsive design elements to ensure a pleasant and accessible experience across devices, with dynamic UI adjustments like showing time quantum or priority settings only when relevant algorithms are chosen.

Functionally, the JavaScript code handles all core operations: generating and validating input tables, implementing each scheduling logic to produce an execution sequence, merging consecutive idle or same-process blocks for clarity, and calculating key performance indicators like finish time, turnaround time, and waiting time. Results are displayed through a responsive Gantt chart that automatically scales to fit the screen without horizontal scrolling, alongside detailed process tables and average statistics. Small user experience features include toast notifications for feedback, smooth scrolling to results, hover effects, and subtle animations, making it both an educational tool and a visually appealing application for learning about CPU scheduling concepts.



V. References

- Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating systems: Three easy pieces*. Arpaci-Dusseau Books.
- Hansen, P. B. (2001). The evolution of operating systems. In P. B. Hansen (Ed.), *Classic operating systems* (pp. 1–34). Springer. https://doi.org/10.1007/978-1-4757-3510-1_1
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.
- Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.